

ORION FRONTEND: FEJLESZTŐI DOKUMENTÁCIÓ

Az Orion egy prémium minőségű állásportál platform, amely a modern webes technológiák legjavát ötvözi. A dokumentáció célja, hogy átfogó képet adjon a frontend architektúráról, a felhasznált technológiákról és a fejlesztési folyamatokról.

Státusz: Verzió 2.6 | **Production Ready** | Utolsó frissítés: 2026. március 30.

Tartalomjegyzék

- Technológiai Stack
- Projekt Felépítés és Architektúra
- Környezeti Beállítások és Telepítés
- Kulcsfontosságú Modulok
- Állapotkezelési Mélymerülés
- API Réteg és Kommunikáció
- Stílus és Design Irányelvek
- Tesztelési Stratégia
- Telepítés és CI/CD
- Külső Függőségek és Alrendszerek
- Elérhető Sriptek
- Komponens Katalógus (Storybook)
- Útvonal Regiszter (Sitemap)
- Védelmi Logika és Szerepkörök
- Layout Integráció

1. Technológiai Stack

Az alkalmazás alapvázát a **React 19** és a **Vite** környezet alkotja. A teljes kódbázis **TypeScript** alapú, biztosítva a típusbiztonságot és a könnyű karbantarthatóságot.

Kategória	Technológia / Könyvtár	Szerepkör
Core Framework	React 19	UI keretrendszer
Build Tool	Vite	Fejlesztői környezet és bundling
Routing	react-router-dom	Oldalak közötti navigáció
Animations	framer-motion	Prémium animációk és transitions
Forms	react-hook-form + zod	Űrlapkezelő és validáció
Database/Auth	@supabase/supabase-js	Backend-as-a-Service integráció
Styling	Vanilla CSS + Lucide icons	Egyedi arculat és ikonkészlet
Visual Effects	tsparticles	Interaktív vizuális elemek

2. Projekt Felépítés és Architektúra

Az Orion a **Feature-Based Architecture** elvet követi. Ez azt jelenti, hogy a kód nem technikai rétegek (pl. components, containers), hanem üzleti funkciók szerint van csoportosítva.

```
src/  
├── Api/           # Típusbiztos API hívások (Application, Jobs, Auth, etc.)  
├── components/    # Megosztott UI komponensek  
│   ├── base/      # Alapszintű elemek (Button, Input)  
│   ├── layout/     # Elrendezési elemek (Header, Footer, Sidebar)  
│   └── ui/         # Komplexebb UI modulok  
├── features/      # Üzleti logikai egységek (Features)  
│   ├── company/    # Céges adminisztráció, ATS, Álláskezelés  
│   ├── user/        # Álláskeresés, Jelentkezések, Messenger  
│   ├── auth/        # Bejelentkezés és regisztráció folyamatok  
│   └── public/      # Mindenki számára elérhető kezdőoldalak  
├── hooks/         # Egyedi React hook-ok (useAuth, useIsMobile)  
├── locales/        # Többnyelvűségi fordítások (i18n)  
└── validation/     # Globális validációs sémák (Zod)
```

3. Környezeti Beállítások és Telepítés

A projekt futtatásához az alábbi előfeltételek szükségesek:

- **Node.js:** Minimum 20.0.0 verzió ajánlott.

- **Csomagkezelő:** A projekt hivatalosan npm alapú (package-lock.json használata kötelező).

Környezeti Változók (.env)

A helyi fejlesztéshez hozzon létre egy .env fájlt az alábbi kötelező kulcsokkal:

```
ENCRYPTION_KEY=... # Adattitkosítási kulcs
JWT_SECRET=... # Hitelesítési token titka
VITE_API_URL=... # Backend API címe (alapértelmezett:
http://localhost:4000)
FRONTEND_API_URL=... # Frontend címe (alapértelmezett: http://localhost:5173)
SUPABASE_SERVICE_KEY= # Supabase hozzáférési kulcs
```

4. Kulcsfontosságú Modulok

4.1. Applicant Tracking System (ATS)

A munkaadók számára kifejlesztett vizuális jelöltkezelő rendszer. Lehetővé teszi a jelentkezők státuszának módosítását, önéletrajzuk megtekintését és az azonnali üzenetküldést. A **Framer Motion** animációk révén a kártyák mozgatása és a státuszváltás folyamatos élményt nyújt.

4.2. Orion Messenger

Valós idejű kommunikációs platform, amely integrálódik az ATS-be. Automatikusan társítja a beszélgetéseket a jelöltekhez és álláshirdetésekhez, megkönnyítve a toborzási folyamatot.

Fontos: Az üzenetküldéshez aktív felhasználói munkamenet szükséges, amelyet a MessengerProvider felügyel globalisan.

5. Állapotkezelési Mélymerülés

Az Orion hibrid megoldást alkalmaz az adatok kezelésére, elválasztva a szervertől a lokális felhasználói élménytől.

5.1. Szerver Állapot (Server State)

Jelenleg az alkalmazás standard `fetch` hívásokat és `useEffect` hook-okat használ az API rétegen (`src/Api`) keresztül. A jövőbeni tervek között szerepel a **TanStack Query (React Query)** bevezetése a hatékonyabb gyorsítótárazás (caching) érdekében.

5.2. Globális Kliens Állapot

A központosított logikát (pl. Messenger munkamenet, felhasználói autentikáció) natív **React Context API** vezérli (pl. `MessengerProvider`). Ez biztosítja a "prop-drilling" elkerülését anélkül, hogy külső könyvtárat (mint a `Redux` vagy `Zustand`) kellene használni, így csökkentve a bundle méretét.

6. API Réteg és Kommunikáció

Minden hálózati kérés egy központosított API rétegen keresztül zajlik, amely TypeScript interfészekkel garantálja a válaszok formátumát.

Szolgáltatás	Típus	Funkció
<code>authApi.ts</code>	AUTH	Login, Register, Password Reset
<code>advertisementApi.ts</code>	JOBS	Állás hirdetések listázása és módosítása
<code>applicationApi.ts</code>	ATS	Jelentkezések leadása és státuszuk kezelése
<code>messageApi.ts</code>	MSG	Üzenetek küldése és lekérése

7. Stílus és Design Irányelvek

A vizuális megjelenés a "Modern Professional" irányzatot követi. A technikai megvalósítás alapja a **Vanilla CSS**, globális változók használatával.

7.1. Design Tokens (`index.css`)

Az arculati elemek központosítva vannak a `:root` elemben:

- `--orion-blue: #2c5aa0;` - Elsődleges márka szín
- `--bg: #f8fafc;` - Világos hátterek
- `--panel: #ffffff;` - Kártyák és panelek
- `--text: #0f172a;` - Tipográfiai alapszín

7.2. CSS Architektúra

A komponensek saját CSS fájlokkal rendelkeznek a jobb modulárisság érdekében. A jövőbeni skálázhatóság érdekében javasolt a **BEM (Block Element Modifier)** elnevezési konvenció szigorúbb követése vagy **CSS Modules** alkalmazása.

8. Tesztelési Stratégia

Roadmap: Jelenleg az automatizált tesztelés implementálása folyamatban van. A "Production Ready" státuszhoz az alábbiak bevezetése tervezett:

- **Unit & Integration:** Vitest használata a logikai hook-ok és segédfüggvények tesztelésére.
- **End-to-End (E2E):** Playwright integráció a kritikus folyamatokhoz (Messenger, Jelentkezés leadása).
- **Visual Regression:** Chromatic használata a komplex UI animációk és Glassmorphism hatások ellenőrzésére.

9. Telepítés és CI/CD

A frontend telepítése statikus build alapú, így bármilyen modern hosting szolgáltatón (Vercel, Netlify, AWS S3) futtatható.

- **Branching Strategy:** A fejlesztés a GitHub Flow modellt követi (feature ágak, Pull Request alapú merge).
- **CI Pipeline:** Javasolt a GitHub Actions használata a build és lint folyamatok automatizálására minden feltöltésnél.

10. Külső Függőségek és Alrendszerek

A frontend teljeskörű működéséhez az alábbi alrendszerek párhuzamos futtatása szükséges:

10.1. Local Backend (Express)

A frontend az adatokat a helyi Express szervertől kapja, amely a `server.ts`-en keresztül kommunikál az adatbázissal.

10.2. OrionAI (Python Module)

Az intelligens funkciókért (pl. CV elemzés) felelős Python modul. Futtatásához **Python 3.10+** és dedikált virtuális környezet (.venv) szükséges.

11. Elérhető Scriptek

A projekt indításához az alábbi `npm` parancsok használhatóak:

```
# Fejlesztői szerver indítása
npm run dev

# Backend szerver indítása
npm run server

# OrionAI (Python) indítása
npm run OrionAI

# Production build készítése
npm run build
```

12. Komponens Katalógus

A projekt Feature-Based Architecture-t használ megosztott UI komponensekkel a `src/components` könyvtárban. A jelenleg elérhető megosztott komponensek:

Újrahasználható UI Elemek (`src/components/ui`)

- `Button` – Egységes gombstílus az egész alkalmazásban
- `InputField` – Formázott beviteli mező label és validálás támogatással
- `TextArea` – Többsoros szöveges beviteli mező
- `Checkbox` – Egyedi jelölőnégyzet komponens
- `ConfirmModal` – Visszaigazoló modális ablak (pl. törlés előtt)
- `AnimatedNumber` – Animált számláló vizuális statisztikákhoz

Roadmap – Storybook: Jelenleg nincs vizuális komponens katalógus. A jövőbeni tervek között szerepel a **Storybook** integrációja, amely lehetővé teszi a `base` és `ui` komponensek böngészését és tesztelését az alkalmazás futtatása nélkül.

13. Útvonal Regiszter (Sitemap)

Az összes útvonal az App.tsx fájlban van deklarálva. Az alábbi táblázat tartalmazza a teljes alkalmazás URL térképét és a hozzájuk tartozó hozzáférési szinteket.

URL Útvonal	Hozzáférés	Leírás
/	PUBLIC	Nyilvános kezdőoldal (Landing Page)
/UserLoginPage	GUEST	Felhasználói bejelentkezés
/UserRegisterPage	GUEST	Felhasználói regisztráció
/user/verify	PUBLIC	E-mail cím megerősítése
/user/sendverify	PUBLIC	Megerősítő e-mail újraküldése
/user/password/reset	PUBLIC	Jelszó-visszaállítás igénylése
/user/password	PUBLIC	Új jelszó mentése
/userhomepage	USER	Felhasználói kezdőlap
/listjobs	USER	Összes állás böngészése
/job/show/:id	USER	Kiválasztott állás megtekintése
/favorites	USER	Kedvenc állások
/JobApplications	USER	Leadott jelentkezések
/uploadresume	USER	Önéletrajz feltöltése
/usereditprofile	USER	Felhasználói profil szerkesztése
/messenger	PUBLIC	Üzenetküldő felület
/CompanyLoginPage	GUEST	Cég bejelentkezés
/CompanyRegisterPage	GUEST	Cég regisztráció
/company	COMPANY	Cég kezdőlap / dashboard
/ShowListedJobs	COMPANY	Meghirdetett állások listája

URL Útvonal	Hozzáférés	Leírás
/AddJob	COMPANY	Új álláshirdetés létrehozása
/company/edit/:id	COMPANY	Álláshirdetés szerkesztése
/company/ATS/:id	COMPANY	Jelöltkövetési rendszer (ATS)
/CompanyEditProfile	COMPANY	Cégprofil szerkesztése
/employees	COMPANY	Alkalmazottak kezelése

14. Védelmi Logika és Szerepkör-alapú Hozzáférés

A hozzáférés-védelem az `IsLoggedIn` komponensen keresztül valósul meg (`src/IsLoggedIn.tsx`), amely egy általános célú **Auth Guard**-ként működik. Minden védett útvonalat ezzel a komponenssel burkolnak be az `App.tsx`-ben.

Működési logika

Az `IsLoggedIn` komponens a `mode` prop alapján háromféle viselkedést implementál:

Mode	Feltétel	Átirányítás
<code>mode="user"</code>	Csak bejelentkezett felhasználók érhetik el	Ha nincs bejelentkezve → <code>/UserLoginPage</code>
<code>mode="company"</code>	Csak bejelentkezett cégek érhetik el	Ha nincs bejelentkezve → <code>/CompanyLoginPage</code>
<code>mode="guest"</code>	Minden Felhasználó elérheti	Ha be van jelentkezve → a saját kezdőoldalra

Szerepkör cross-redirect logika

Az `IsLoggedIn` nemcsak az autentikációt ellenőrzi, hanem a **szerepkört** is. Ha egy céges felhasználó megpróbál egy `mode="user"` végpontot elérni, azonnal a `/company` kezdőlapra irányítódik át (és fordítva). Az autentikációs állapot a backend `checkAuthStatus()` API hívással ellenőrződik, és a `auth-changed` window esemény hatására automatikusan frissül.

Implementáció: Az összes szerepkör-ellenőrzés egyetlen, újrahasználgató komponensben van központosítva (`IsLoggedIn.tsx`), így az új vegyes szabályok hozzáadása minimális kódváltoztatással jár.

15. Layout Integráció

A layout komponensek (Header, Footer, BannerKicker, stb.) az egyes feature oldalak **belső sablonjaként** vannak beágyazva – nem egy globális wrapper veszi körül az összes útvonalat.

Layout stratégia oldalanként

- **Public Landing Page (/):** Saját, testreszabott Header és animált háttér (`tsparticles`). Nem egyezik meg a belső alkalmazás Headerével.
- **Autentikációs oldalak (Login, Register):** Minimális layout – a fókusz az űrlapon van.
- **Felhasználói és Céges oldalak (védett útvonalak):** Teljes alkalmazás-Header (`LanguageSelector`, `ProfileDropdown`, `MessageDropDown` integrálva), Footer és reszponzív Sidebar navigáció.
- **ATS és Admin felületek:** A belső dashboard Header-t alkalmazzák, kiegészítve a visszavigálási vezérlőkkel (pl. vissza a hirdetések listájához).